TheoryBERT: Translating and Understanding Formal Languages with BERT

Abstract - Natural-language interfaces can produce plausible formal-language answers without establishing that the generated objects represent the requested language or that their visualizations explain the current input. TheoryBERT addresses this translation and understanding gap through a BERT-centered web system for theory of computation and compiler education. A shared contextual encoder supports a typed regular-language plan decoder with learned literal copying and a separate grammar-production decoder. Exact automata construction, reference equivalence checking, production-level derivation checks and arithmetic AST/IR validation connect the learned interpretation to executable formal objects. Interactive views expose the actual machine, grammar, derivation and input-specific behaviour with explicit evidence status. The final DFA model attains 95.32% first-candidate equivalence on 641 generated test requests and 71.67% on 120 frozen fresh-language requests. Grammar and compiler evaluations use task-specific criteria. The approach integrates language interpretation, formal execution and educational representation while retaining explicit limits on English understanding and supported language classes.

Index Terms - BERT, deterministic finite automata, semantic parsing, context-free grammar, compiler education, formal verification.

I. INTRODUCTION

A. Problem Statement

Central problem statement: The inability of NLP systems to reliably support the translation and understanding of formal-language objects and their behaviour. In this work, the statement identifies a reliability gap in an integrated educational setting. It does not imply that prior NLP research has never translated language into programs or regular expressions. The unresolved need is to connect an informal requirement to an appropriate formal object, execute that object correctly and explain its behaviour through an inspectable representation.

A learner may describe binary words ending in 0101, yet the required answer is an infinite language represented by finite memory. Understanding that answer requires more than a graph: the learner needs the meaning of each remembered suffix, a total transition function, accepting conditions and a trace for the supplied word. Similarly, a grammar must expose recursive productions and their derivations, while compiler exercises must connect accepted expressions with syntax trees and symbolic instructions.

TheoryBERT treats these requirements as one translation-and-understanding task. DFA, CFG and compiler workspaces are related views over model-produced formal structure. BERT interprets the description; explicit mathematical components give that interpretation executable meaning and make its consequences visible.

The gap becomes visible when an interface presents a generic diagram for unrelated requirements or labels a plausible candidate as correct without semantic evidence. Mathematical representation must follow the language specification. Execution must follow the particular input. A DFA should keep the same state graph when only its test word changes, while its path and acceptance result change. A context-free derivation tree may grow for a longer input even though its grammar stays fixed.

Correctness has several distinct meanings. A complete transition table may be structurally valid but semantically wrong. A trace may faithfully follow that wrong table. Equivalence to an independent reference DFA is a stronger, decidable claim, but the reference itself must represent the intended requirement. With arbitrary English alone, there is no independent mathematical oracle. The product must preserve these distinctions instead of labelling every successful prediction as verified. This separation is especially important in an educational interface, where a confident graphic can conceal a misunderstood quantifier or boundary condition.

B. Existing Approaches

Rule-driven interfaces can map a small set of phrases to known automata constructors. They are efficient within their accepted syntax, but extending the vocabulary or adding a new composition often requires another handwritten linguistic condition. The project retains such a baseline for controlled evaluation, while the learned application path does not invoke it. Manual automata and grammar entry remain useful for direct formal exercises; they are labelled as supplied objects and do not count as learned language understanding.

Neural semantic parsing learns mappings from descriptions to executable forms. Earlier natural-language regular-expression work established both semantic-unification and learned translation approaches [1], [2]. A direct neural DFA head has a simple output contract: predict the number of states, accepting-state bits and transition destinations. However, every additional state introduces many coupled decisions. A long literal or a conjunction can fail even when the model has recognized the general language family. Training on more templates can improve familiar wording without resolving this structural burden.

C. Proposed Solution and Objectives

The proposed solution uses BERT as the language-understanding basis and introduces task-specific formal targets. For regular languages, the encoder supplies contextual token features to a compact typed plan decoder. A plan such as ENDS(0101) expresses the intended condition explicitly; learned span pointers preserve the literal. An exact compiler constructs a DFA from the predicted plan. The system also implements a direct structured DFA head as an alternative model interface, with the same downstream contract.

This representation gives the neural component fewer dependent graph decisions for common regular-language requirements. It also makes literal identity explicit: the pattern 00101 must keep its leading zeros. Type constraints permit only well-formed operator arguments, while formal bounds prevent uncontrolled products or very large machines. These constraints establish syntactic and structural discipline. They do not guarantee that the predicted operator or numeric argument matches the English request.

The objectives are reliable formal translation within a declared scope, transparent execution of the resulting objects and a visualization whose mathematical content matches the current request. Resource-conscious training and shared encoder reuse support practical operation. Accuracy, formal validity, response latency and adaptation cost remain separate measures because none can stand in for the others.

For CFG and compiler modes, BERT predicts grammar-production tokens through a separate decoder over the same evaluated encoder. Earley parsing then checks membership of a particular token sequence. Production checks validate each returned derivation and its yield. The arithmetic compiler lowers an appropriate checked parse to an AST and three-address-style symbolic instructions. These are explicit formal operations over BERT-produced structure. They do not use the automata plan vocabulary to pretend that all context-free languages are regular.

The complete system makes formal-language objects inspectable through an English requirement, a model-produced specification, an executable object and evidence-backed behaviour. Versioned corpora, canonical-language splits and checkpoint fingerprints connect the implementation to reproducible evaluation. The purpose of this methodology is to assess the final solution and its limits rather than present a sequence of development changes.

D. Correctness and Educational Representation

A useful educational answer should reveal the model interpretation and its consequences. For example, an exact-word request for 00101 must reject every proper extension even if that extension contains the pattern. A substring request must accept those extensions. The two machines can share visual conventions while differing in their state meanings and accepting behavior. Such minimal semantic differences should be evaluated directly rather than hidden inside a broad average score.

The same principle applies to a numeric requirement. Exactly two ones, at most two ones and an even number of ones are different languages, despite shared words and symbols. An interpretation error may produce a complete machine with no malformed transitions. Therefore the final output needs both an inspectable object and task-appropriate semantic evidence. A displayed sample of accepted words is helpful, but agreement on a finite sample cannot replace exact equivalence when a reference DFA is available.

The system also distinguishes a language specification from an execution specimen. The graph is chosen by the former; the highlighted route is chosen by the latter. This avoids teaching the misconception that each character in an input should become a new DFA state. In grammar mode, however, a longer word may require a larger derivation tree while the productions remain fixed. The application must preserve that mathematical difference instead of forcing every task into one visual template.

Translation, formal execution and representation are consequently three obligations of the same underlying problem. A failure at any boundary can mislead the learner: an incorrect interpretation yields the wrong language, an incorrect simulator yields the wrong behaviour, and a stale visualization conceals both. The architecture exposes these boundaries without treating DFA, grammar and compiler support as unrelated problem statements.

E. Organization of the Paper

Section II reviews related work and identifies the translation-and-understanding gap. Section III describes the component architecture and algorithms. Section IV states the key contributions. Section V documents datasets and provenance. Section VI evaluates the final system. Section VII presents the web interface, visualization and resource design. Section VIII discusses limitations and future work, and Section IX concludes the paper.

II. LITERATURE REVIEW

A. Language Representations and Domain Adaptation

Vaswani et al. introduced the attention-based Transformer architecture [3]. BERT supplies the bidirectional representation foundation used here [4]. RoBERTa examines how pretraining choices and data affect comparison [5], motivating caution about unmatched conditions. TheoryBERT retains its pinned BERT encoder rather than silently substituting RoBERTa. Its downstream head must still learn a formal interpretation; attention alone does not specify automata states or grammar productions.

Tai et al. study constrained-resource vocabulary extension through exBERT [6]. BioBERT provides a domain-pretraining example in biomedicine [7], while Weber et al. describe token-augmented fine-tuning [8]. These support studying adaptation choices, not assuming every extension improves correctness. TheoryBERT is exBERT-inspired, with a residual module after the backbone rather than an exact reproduction. Its direct V3 model updates two BERT blocks; the new plan

experiment freezes that evaluated encoder.

These adaptation studies inform the choice to retain pretrained language representations while learning explicit formal targets. Vocabulary adaptation can improve domain representation, but it does not itself define a correct automaton or derivation. The present solution therefore combines contextual encoding with typed semantic output and independent formal checks. The role of NLP is interpretation; the role of the formal component is precise execution of the interpretation.

B. Natural Language to Regular Expressions

Kushman and Barzilay connect natural-language requests with regular expressions using semantic unification [1]. An important lesson for this implementation is that surface-form equality is inadequate: distinct formal expressions can denote the same language. TheoryBERT uses exact DFA equivalence for its automata metric and canonical language identity for split grouping. Its neural architecture differs, but the evaluation should still respect semantic equivalence rather than rewarding one arbitrary spelling of a regular expression.

Locascio et al. investigate learned translation from natural language to regular expressions and provide a public code/data resource [2], [9]. TheoryBERT imports only a restricted portion of the KB13 mirror, because the local task is binary and its regex compiler deliberately supports a bounded dialect. Published performance on the original task cannot be compared directly with the present filtered, generated benchmark. Dataset reuse is therefore accompanied by revision-pinned links, filtering counts and an explicit distinction between source pairs and retained local examples.

C. Intermediate Representations and Compositionality

Herzig et al. examine how intermediate representations affect compositional generalization in pretrained sequence-to-sequence semantic parsers [10]. Their findings motivate investigating a target representation aligned more closely with a request. The TheoryBERT plan experiment applies that design idea to a different architecture and formal domain: a frozen BERT encoder, a compact parallel decoder and an automata compiler. It is an adaptation of an established research direction, not a reproduction of their system or an assertion of comparable benchmark performance.

The local representation makes the distinction concrete. A suffix requirement can be expressed by one operator and a literal rather than a separately predicted destination for every state and alphabet symbol. A conjunction can preserve two interpretable subexpressions before a product construction combines their languages. This reduces output entanglement for covered families. It also restricts expressiveness: a complete arbitrary transition system may not have a short expression in the chosen vocabulary. Structural reconstruction and typed-plan prediction are related tasks with different coverage.

Oren et al. study contextual representations, alignment supervision and template balance in compositional parsing [11]. Their analysis motivates literal pointers and composition-specific evaluation here. TheoryBERT does not implement all their mechanisms. Its uneven generated operator distribution also prevents an aggregate score from establishing systematic generalization.

A particularly important distinction is between a new wording and a new formal target. Shared templates can make a language-held-out test easier than an unrestricted human-written request. Conversely, a new wrapper can expose phrasing sensitivity without testing a new operator. The frozen v2 comparison contains protected formal language groups and a distinct wording wrapper; it is still generated from the same plan family catalogue. Its purpose is a controlled local comparison, not an unrestricted natural-language benchmark.

D. Constrained Decoding

PICARD constrains autoregressive generation by rejecting tokens that do not satisfy a target-language parser [12]. The connection here is the general principle that formal output constraints should be enforced during decoding. TheoryBERT uses a bounded typed prefix search over a small operator vocabulary, with literal spans restricted to binary text. Its head predicts all output positions in parallel. It does not use PICARD software, its incremental SQL parser or its T5 training protocol.

A constrained decoder can still make semantic errors: ENDS(01) and STARTS(01) are both valid plans. Evaluation measures first-candidate equivalence before reference-assisted selection and reports within-five success separately. A checker-selected answer is not the model's unaided first prediction.

E. Behavioral Testing and Formal Execution

Ribeiro et al. propose CheckList to expose model failures through behavioral testing beyond a single held-out accuracy value [13]. This motivates targeted tests for negation, boundaries, literal substitution and combinations of requirements. TheoryBERT keeps known product fixtures, a retired mixed challenge and a newly frozen comparison. A fixture is a regression obligation once it has influenced development. It cannot also serve as independent confirmation that a newly trained model generalizes.

Classical automata and parsing methods supply the executable semantics after language analysis. Cornell's compiler notes explain total DFA transitions and the connection between regular specifications and lexical analysis [14]. NLTK documents incremental Earley chart parsing [15]. TheoryBERT uses these established principles to validate structures, simulate words and check derivations. The project contribution is the integration and evidence discipline, rather than a new DFA equivalence or general CFG parsing algorithm.

Exact DFA equivalence uses product exploration. General CFG equivalence is not supplied. A parse verifies membership under one grammar, not all-word correspondence with English. Preserving an AST in symbolic intermediate code is likewise a bounded claim, without a full-language optimizer or

target-machine execution.

F. Research Gap and Positioning

The research gap is the absence of a sufficiently reliable, inspectable connection between NLP interpretation and the representation of formal-language behaviour in one educational workflow. Existing semantic parsers demonstrate translation, while automata and compiler tools demonstrate formal execution. TheoryBERT connects those capabilities through typed contracts, task-specific verification and a shared interface that explains the current formal object and input.

Formal-language research also cautions against extrapolating finite accuracy to unrestricted capability. Deletang et al. empirically study neural generalization across the Chomsky hierarchy [16]. Strobl et al. survey transformer expressivity under differing architectural assumptions [17]. TheoryBERT instead predicts bounded formal specifications, then executes exact algorithms. Its frozen wording evaluation tests the complete approach, without claiming a reproduction of either theoretical analysis or universal language recognition.

The approach combines established pretrained representations, intermediate semantics, formal constraints and behavioral testing. It claims no first invention, universal correctness or superiority to large language models. Human-authored evaluation, matched-data ablations and several seeds remain necessary.

III. PROPOSED SOLUTION

A. Component Architecture

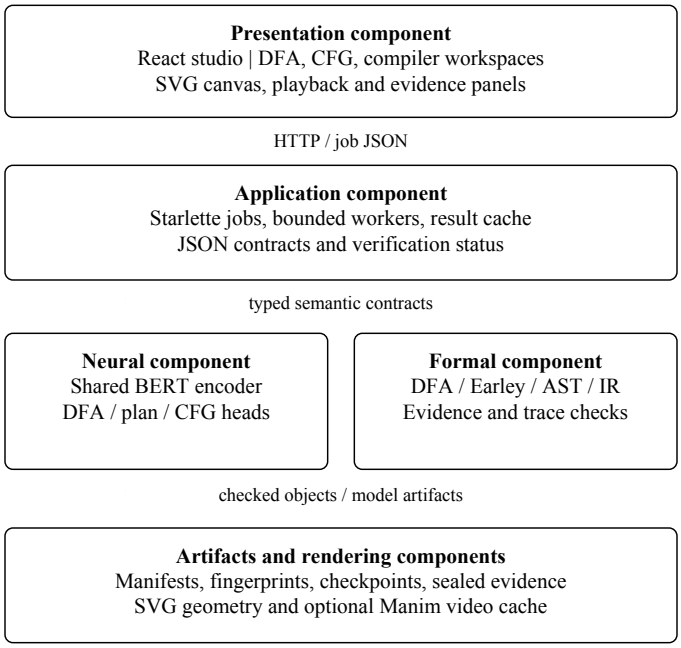


Fig. 1. Component architecture and named formal data interfaces.

The browser owns presentation and playback. Starlette owns model loading, jobs, validation and result evidence. A shared BERT encoder serves the direct DFA predictor and the small grammar or semantic-plan heads. The formal component accepts typed objects, not raw English. It supplies automata construction, simulation, equivalence, CFG parsing and symbolic compiler checks. A separate rendering component consumes these checked structures and their traces. Dataset manifests and model fingerprints provide the artifact boundary.

This organization avoids a second language interpreter in the renderer. Changing an English description invokes the selected BERT predictor. Changing a test word reuses the formal object and computes another trace. The SVG geometry, node styles, edge routing and playback controls remain the same when a different prediction architecture is selected. The neural and formal components share a versioned JSON DFA contract, so an improved interpreter need not redesign the final visual representation.

B. Structured DFA Interface

The implemented direct interface predicts a state count, accepting-state bits and binary transition destinations from a pooled BERT representation. It supports up to eight states and constrains destinations to the active range. The semantic-plan interface supplies larger supported constructions without requiring the network to predict every transition independently. Both interfaces emit the same complete DFA JSON contract, allowing the formal checker, simulator and renderer to operate independently of model architecture.

C. BERT Semantic Plan Decoder

The semantic-plan head projects contextual BERT features from width 768 to 128. Twenty-four learned output queries attend through two Transformer decoder layers, with four attention heads and feed-forward width 256. A token classifier predicts operators, constants and termination. Separate start and end projections score literal boundaries against encoder tokens. A single-bit pattern uses a learned fixed output class; longer patterns use the learned copy action and token-span scores.

The typed vocabulary includes universal and empty languages; prefix, suffix, substring and exact-word requirements; Boolean negation, conjunction and disjunction; count equality and bounds; and count or length modulo conditions. Numbers are restricted to 0 through 16. The compiler accepts binary literals up to 32 characters and at most 64 final states. The generated training catalogue covers patterns up to eight characters and moduli 2 through 8. Implementation capacity must not be confused with demonstrated training coverage.

Training minimizes output-token cross-entropy plus weighted start/end pointer cross-entropies at copy positions. Padding is excluded. The encoder is immutable and fingerprinted for head compatibility. A packed real-token feature cache supports bounded-memory adaptation. Canonical-language groups separate training, validation and testing. Validation selects the checkpoint; final test partitions are excluded from gradient updates and selection.

At inference, a bounded beam tracks the argument types still required by the predicted prefix expression. It ranks up to three token alternatives for a type and up to two literal spans for copying, with beam width eight. Search uses relative neural score gaps and deduplicates completed canonical DFAs. The formal compiler rejects invalid arguments and bounds. There is no English keyword matcher, reference-guided plan construction or replacement of an incorrect prediction with a hand-constructed answer.

D. Formal and Grammar Contracts

Primitive plans compile to exact automata. Negation complements a complete DFA; conjunction and disjunction use reachable product construction. A 256-state intermediate-product limit precedes final minimization. Product BFS compares candidate and reference languages and returns a shortest distinguishing word on disagreement. An empty list is the empty-word witness; a missing witness has a different meaning.

For CFG tasks, the separate BERT head predicts production tokens. Parsing validates each production and terminal yield before a tree is released. Arithmetic lowering verifies temporary definitions, operator arity, operand order and reconstruction of the AST expression. These guarantees concern the predicted formal objects. Correct interpretation of the original English still depends on the learned model and an appropriate evaluation reference.

IV. KEY CONTRIBUTIONS

1) BERT-centered formal translation. A shared contextual encoder supports typed regular-language plans, learned literal copying and grammar-production decoding. Operator selection and argument interpretation come from BERT features. Formal constructors execute model-produced specifications instead of independently recognizing English phrases.

2) Understanding through executable representation. A common web interface connects the predicted language to states, transitions, input traces, production derivations and arithmetic AST/IR. The displayed structure is generated from the actual formal object, and execution highlights are derived from the current input.

3) Explicit correctness boundaries. Structural validation, reference DFA equivalence, derivation validity and compiler consistency are represented as distinct claims. Unreferenced English predictions remain inspectable candidates. Sealed evidence gates video export and prevents a polished visualization from implying unsupported semantic certainty.

4) Reproducible end-to-end implementation. Versioned datasets, canonical-language partitions, pinned source revisions, shared encoder loading and resource guards support a complete deployable system. Model evaluation and application integration checks assess the solution at their respective boundaries.

These are integration and evaluation contributions. Pretrained encoders, intermediate representations, constrained decoding and classical formal algorithms have prior art. No first-invention or state-of-the-art claim is made.

V. DATASETS AND PROVENANCE

A. External Sources

The downloaded pretrained artifact is google-bert/bert-base-uncased [18], pinned to revision 86b5e0934494bd15c9632b12f734a8a67f723594. BookCorpus and English Wikipedia are inherited pretraining sources described by the model card; the project did not download or retrain on those raw corpora. The source artifact license is Apache-2.0.

The external supervised source is the KB13 description/regex mirror in DeepRegex [9], pinned to revision 096490db7f4b0394fbb46b914cb35a0aa1cba29c. Of 824 paired rows, 55 pass local import restrictions; 497 use unsupported regex dialect, 271 become constant languages after binary restriction and one exceeds a repeat bound. Deduplication leaves 47 public-regex rows in the V3 corpus. The supplied dataset license is MIT. These figures describe this import, not original benchmark coverage.

B. Local Supervision and Evaluation Sets

TABLE I. Local corpus versions. Versions overlap and must not be summed as independent data.

Corpus	Rows	Train / val / test
Shipped sample	261	181 / 39 / 41
English DFA v1	1,785	1,363 / 213 / 209
Expanded DFA	3,207	2,437 / 391 / 379
DFA V3	6,275	4,889 / 745 / 641
Robust DFA candidate	6,738	4,924 / 1,173 / 641
Grammar original	1,299	999 / 170 / 130
Grammar robust	1,652	1,267 / 255 / 130
Semantic plan v1	11,415	9,187 / 1,171 / 1,057
Fresh plan v2	254	0 / 134 / 120

The shipped sample contains 96 unique languages. English DFA v1 and its expanded version contain 208 each. V3 contains 696, including substring, prefix, suffix, modulo, literal, negation, bounded-count and edge cases. The robust direct candidate contains 1,468. The original grammar corpus has 123 distinct grammars; robust merging yields 119 after canonicalization and deduplication. Arithmetic grammars are training/product fixtures, so their preset success is not held-out grammar generalization.

The plan dataset contains 2,914 canonical DFA languages. It annotates 6,274 prior V3 rows with copyable equivalent plans and adds generated supervision; one prior row has no supported copyable annotation. Languages inherit protected prior split roles, and new identities use a deterministic hash split. Supplementary challenge descriptions contribute supervision only when their canonical language belongs to

training. Assertions enforce one partition per language.

The challenge assets cover DFA, CFG and compiler requests with different wording and boundary conditions. The 165-case mixed suite supplies regression coverage across all three workspaces. The frozen 254-case plan suite contains 134 validation and 120 test languages outside plan training. A 57-case test subset also satisfies the direct interface bounds of eight states and 64 input tokens. These assets are not independent corpora to be added to training counts.

The companion report lists every corpus, its role, counts, hashes, generation code and source links. The dataset provenance archive distributes the permitted local supervision and manifests. The encoder and task heads are published separately as a versioned inference bundle. Generated data is identified as generated; inherited pretraining sources are distinguished from data actually used in local adaptation.

VI. EVALUATION OF THE FINAL SOLUTION

A. Evaluation Questions and Metrics

The evaluation asks whether the system translates descriptions into the intended supported formal objects, whether formal execution is correct and whether the interface presents the resulting behaviour faithfully. These questions correspond to one integrated problem but require different evidence. A first-candidate DFA is scored by exact equivalence to an independent reference. Invalid and unsupported outputs count as incorrect. Within-five success reports reference-assisted candidate selection separately.

A formal checker cannot establish the intended meaning of arbitrary English by inspecting a prediction alone. The independent DFA reference supplies a semantic target for measured automata tasks. Without it, structural validation and simulation remain useful but narrower claims. Grammar evaluation records canonical production exactness and checked derivations. Compiler fixtures check both valid-expression lowering and correct rejection of invalid expressions.

Canonical DFA identities group equivalent languages before splitting. The final plan corpus has 2,914 distinct identities. Frozen fresh-language test requests are outside training and do not determine checkpoint selection. Previously inspected mixed cases are reported as regression evidence. Dataset versions overlap, and results from different partitions are not pooled into a single general accuracy claim.

B. Final DFA and Grammar Results

The final plan decoder produces 611/641 first-candidate equivalent DFAs on the generated regression test, giving 95.32% exact equivalence. On the 120-case frozen fresh-language test it produces 86 correct first candidates, giving 71.67%. The 57-case capacity-matched subset yields 38/57, or 66.67%. These values measure supported semantic translation and show a meaningful remaining gap between generated and fresh wording.

The grammar decoder produces 130/130 exact canonical production outputs on its generated test set. The inspected mixed grammar suite yields 10/14 successful cases, and the compiler fixture subset yields 9/9 expected outcomes, including negative cases. The small mixed subsets do not establish unrestricted CFG or compiler accuracy. Production exactness and grammar-language equivalence are distinct.

C. Final System Measurements

TABLE II. Final task-specific evaluation. Reference-assisted search is not unaided model accuracy.

Evaluation asset	Measure	Result
Generated DFA test	First equivalence	611/641; 95.32%
Fresh DFA test	First equivalence	86/120; 71.67%
Matched fresh subset	First equivalence	38/57; 66.67%
Fresh DFA test	Within five	119/120; 99.17%
Generated grammar	Production exactness	130/130
Mixed CFG fixtures	Expected result	10/14
Compiler fixtures	Expected result	9/9

The semantic head contains 537,385 trainable parameters over a shared frozen BERT encoder. Its recorded adaptation run takes 230.8 seconds including feature extraction and head training. The encoder still executes at inference, and typed search and automata construction add work. These resource measurements describe the implementation; they are not the central research contribution or a substitute for correctness.

The frozen fresh test reveals uneven language coverage. Exact-word requests yield 23/23 correct first candidates, substring requests 21/21 and prefix requests 20/22, while suffix requests yield only 1/18 under that wording distribution. Aggregate scores therefore cannot justify a claim that every supported operator is reliably understood. Failure analysis remains part of the final solution assessment.

Integration verification exercises the real HTTP job boundary, changed input traces, grammar generation, rejected expressions and MP4 export. The completed suite contains 105 passing tests, and the React production build succeeds locally and in GitHub Actions. Browser checks assess dynamic graph content and responsive layout separately from model scores. A working endpoint or attractive diagram is not semantic evaluation.

The results support a bounded NLP-to-formal workflow with inspectable execution. They also show why exact verification and honest preview labels remain necessary. A derivation proves membership under the displayed grammar, and an equivalent DFA proves agreement with its reference. Neither claim automatically proves that an arbitrary English requirement has been interpreted correctly.

VII. WEB SYSTEM AND RESOURCE CONTROL

A. Website and Interaction Design

The React/Starlette product provides DFA, CFG and arithmetic workspaces, settings and research information. Reusable SVG components display server-produced formal objects. Python owns the representations and verification decisions; the browser draws their content and controls playback. The project source is identified in [19].

DFA mode accepts an English requirement, a binary word and a prediction source. Learned mode uses the active evaluated checkpoint. Demonstrations, incorrect fixtures and supplied JSON retain distinct provenance. Benchmarks can supply independent references; free-form requests receive structurally checked previews without an equivalence claim.

The graph displays the actual states, total transitions and accepting set. Playback follows the current input trace. Access strings, accepting completions, visits and example words are computed from that machine. Changing only the input changes execution while preserving the language graph.

CFG mode shows predicted productions, parse alternatives and checked derivations. Character, terminal-token and arithmetic-lexeme modes have explicit contracts. The compiler preserves original lexemes in its AST and displays derived symbolic instructions. Syntax failures retain the actual model-produced grammar for inspection.

B. API and Evidence Boundaries

The API supplies health, catalog, research, bounded asynchronous jobs, simulation and cached-video routes. Model caching lets grammar and plan heads share their fingerprinted immutable encoder. The server owns result evidence, provenance and revision invalidation.

Optional Manim export requires sealed DFA equivalence evidence or a production-checked tree. The renderer rechecks evidence and reconstructs traces. An unverified prediction remains an SVG preview with a clearly narrower claim.

C. Input-Dependent Representation

The visualization consumes server-produced DFA or tree objects through a common representation contract. Node count, transitions, accepting roles, state visits and highlighted execution come from the current result. Diagram styling and playback controls are reusable presentation components. Mathematical content is regenerated from the object rather than selected from a catalogue of display templates.

Presentation stays stable while content changes. A longer suffix can require more states; a longer input can revisit existing states without adding nodes. Tests compare visible counts with the formal object and playback with simulation, guarding both graph construction and execution fidelity.

D. Compute, Storage and Reproducibility

The run uses one heavy job, two CPU threads and reduced scheduling priority. Packed real-token float16 features are padded only in small batches, then their cache is removed. Guards record resource snapshots and enforce 3 GiB free disk, 8% available memory and a 0.75 GiB swap-growth allowance.

The training workflow bounds caches, thread count and simultaneous heavy jobs. Frozen encoder features are stored in a packed form and removed after adaptation. Inference heads share compatible encoder weights. CPU memory-mapped loading retains the evaluated FP32 tensors without quantization, reducing anonymous loading copies while preserving model precision.

The deployment image builds React and installs CPU inference, Cairo/Pango and ffmpeg. A versioned GitHub Release supplies a SHA-256-pinned model bundle at build time. Startup requires both active checkpoints; the health endpoint exposes model readiness and deployed commit. This defines the deployment architecture. A cloud memory-limit check separately determines whether a particular hosting tier can support the full runtime.

Pinned source revisions, dataset hashes, encoder fingerprints and per-case evaluation files make the system reproducible. Resource limits support safe local operation, while the public web interface supplies the formal-language workflow. Training efficiency and server capacity remain operational properties; the primary claim concerns translation and understanding of formal objects and their behaviour.

VIII. LIMITATIONS AND FUTURE WORK

The current automata task is binary. Plan training covers a finite operator vocabulary, bounded numbers and selected literal lengths. A valid plan may misunderstand negation, numeric scope, conjunction or a paraphrase outside training. The decoder can abstain on invalid output, but it has no calibrated semantic-confidence guarantee. Exact DFA construction from a plan proves implementation semantics; it does not prove that the plan is the intended interpretation of English.

Evaluation uses generated descriptions and one training seed. Canonical grouping reduces formal-target leakage, but shared operator families and template structure limit extrapolation to unrestricted language. The fresh test has 120 cases, with 57 in the matched subset. A human-authored study, several seeds and calibrated unsupported-intent detection are needed before broader reliability claims.

Grammar exactness depends on canonical productions; general CFG equivalence is not supplied. Compiler support covers symbolic arithmetic, without statements, optimization, machine code or runtime execution. Correct rejection counts in negative compiler cases. These are task boundaries, not general compiler accuracy.

Further work needs matched-data comparisons, several seeds, balanced supervision and calibrated abstention. An editable

predicted plan could expose ambiguity while preserving the diagram. Quantization should require semantic regression checks. Independent human-authored tests and an educational user study remain necessary.

IX. CONCLUSION

TheoryBERT addresses the NLP-to-formal translation and understanding gap through a BERT-centered, executable and inspectable workflow. Typed plans and grammar productions connect natural-language interpretation to exact automata, checked derivations and arithmetic representations. The web system exposes their behaviour through current-object visualization and input-specific execution. Final evaluation demonstrates useful bounded capability while preserving explicit limits on English understanding and formal guarantees.

REFERENCES

[1] N. Kushman and R. Barzilay, "Using Semantic Unification to Generate Regular Expressions from Natural Language," NAACL-HLT, pp. 826-836, 2013. Online source.

[2] N. Locascio, K. Narasimhan, E. DeLeon, N. Kushman, and R. Barzilay, "Neural Generation of Regular Expressions from Natural Language with Minimal Domain Knowledge," EMNLP, pp. 1918-1923, 2016. Online source.

[3] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems, vol. 30, 2017. Online source.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, pp. 4171-4186, 2019. Online source.

[5] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019. Online source.

[6] W. Tai et al., "exBERT: Extending Pre-trained Models with Domain-specific Vocabulary Under Constrained Training Resources," Findings of EMNLP, pp. 1433-1439, 2020. Online source.

[7] J. Lee et al., "BioBERT: a pre-trained biomedical language representation model for biomedical text mining," Bioinformatics, vol. 36, no. 4, pp. 1234-1240, 2020. Online source.

[8] L. J. Weber et al., "Efficient Token Augmented Fine-Tuning," IEEE CSDE, 2023. Online source.

[9] N. Locascio, DeepRegex KB13 dataset mirror, revision 096490db, MIT license. Online source.

[10] J. Herzig et al., "Unlocking Compositional Generalization in Pre-trained Models Using Intermediate Representations," arXiv:2104.07478, 2021. Online source.

[11] I. Oren, J. Herzig, N. Gupta, M. Gardner, and J. Berant, "Improving Compositional Generalization in Semantic Parsing," Findings of EMNLP, 2020. Online source.

[12] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models," EMNLP, pp. 9895-9901, 2021. Online source.

[13] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList," ACL, pp. 4902-4912, 2020. Online source.

[14] Cornell CS 4120, "Automating Lexical Analysis," course notes, Spring 2023. Online source.

[15] NLTK Project, nltk.parse.earleychart: Incremental chart parsing documentation. Online source.

[16] G. Deletang et al., "Neural Networks and the Chomsky Hierarchy," ICLR, 2023; arXiv:2207.02098. Online source.

[17] L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin, "What Formal Languages Can Transformers Express? A Survey," TACL, vol. 12, pp. 543-561, 2024. Online source.

[18] Google BERT / Hugging Face, bert-base-uncased model artifact and model card. Online source.

[19] TheoryBERT project repository, local implementation snapshot, 2026. Online source.